Controller Package

The `controller` package contains the JAX-RS resource classes that define the RESTful API endpoints for the Trouble Ticket management system.

Files

TroubleTicketController.java

This is the primary controller for handling HTTP requests related to Trouble Tickets.

```
@Path("/troubleTicket/v2/troubleTicket")
@Produces(MediaType.APPLICATION_JSON)
@Consumes(MediaType.APPLICATION_JSON)
public class TroubleTicketController {

    @Inject
    private TroubleTicketService service;

    @Context
    private HttpHeaders headers;

    @GET
    public Response findAll(@Context UriInfo uriInfo) {
        MultivaluedMap<String, String> queryParameters = uriInfo.getQueryParameters();
        Map<String, String> filters = new HashMap<>();
        queryParameters.forEach((k, v) -> {
            if (!v.isEmpty()) filters.put(k, v.get(0));
        });

        List<TroubleTicket> list = service.findAll(filters);
        return Response.ok(list).build();
    }

    @GET
    @Path("/{id}")
    public Response findById(@PathParam("id") String id) {
        TroubleTicket ticket = service.findById(id);
        return Response.ok(ticket).build();
    }

    @POST
    public Response create(TroubleTicket ticket) {
        String host = headers.getHeaderString("Host");
        TroubleTicket created = service.create(ticket, host);
        return Response.status(Response.Status.CREATED).entity(created).build();
```

```
    }

    @PATCH
    @Path("/{id}")
    public Response update(@PathParam("id") String id, Map<String, Object> updates) {
        TroubleTicket updated = service.update(id, updates);
        return Response.ok(updated).build();
    }

    @DELETE
    @Path("/{id}")
    public Response delete(@PathParam("id") String id) {
        service.delete(id);
        return Response.noContent().build();
    }
}
```

Line-by-line Explanation

- `@Path("/troubleTicket/v2/troubleTicket")`: Defines the base URI path for all endpoints in this controller.
- `@Produces/Consumes(MediaType.APPLICATION_JSON)`: Specifies that the API communicates using JSON format.
- `@Inject private TroubleTicketService service`: Automatically injects the service layer dependency.
- `findAll(@Context UriInfo uriInfo)`: Handles GET requests, extracting query parameters for filtering.
- `findById(@PathParam("id") String id)`: Retrieves a specific ticket by its ID from the URL path.
- `create(TroubleTicket ticket)`: Handles POST requests to create a new ticket, using the `Host` header to generate the resource's `href`.
- `update(...)`: Handles PATCH requests for partial updates using a Map to represent dynamic fields.
- `delete(...)`: Handles DELETE requests, returning a `204 No Content` status upon success.

Analysis

- **Annotations**:
  - `@Path`: Maps the class or method to a specific URI template.
  - `@Inject`: Standard Jakarta CDI annotation for dependency injection.
  - `@Context`: Injects JAX-RS provider objects like `HttpHeaders` or `UriInfo`.
  - `@PathParam`: Extracts values from the URI path.
- **Data Types**:
  - `List<TroubleTicket>`: Used for returning multiple items because it

maintains order and allows for easy stream processing.
  - `Map<String, Object>`: In the `update` method, a Map is used to capture only the fields provided in the PATCH request body, facilitating partial updates.
- **Architectural Decision**: The controller follows the Thin Controller pattern. It is responsible for request routing, parameter extraction, and response formatting, while delegating business logic to the `TroubleTicketService`.